

```

In [ ]: from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from sklearn.linear_model import LogisticRegression
import numpy as np
# Setting the seed of the random generator
SEED = 42

# Size of the samples in the test set: e.g., 0.2, means that the test set is
composed of 20% of the samples of the entire dataset
test_size = 0.2

# Size of the samples in the validation set: e.g., 0.2, means that the test set is
composed of 20% of the samples of the development set
val_size = 0.2/(1-test_size)

# Outer hold-out
X_dev, X_test, y_dev, y_test = train_test_split(X, y, test_size=test_size,
random_state=SEED)

# Print the size of the development and test sets
print ("The development set size is", "X:", X_dev.shape, "y:", y_dev.shape)
print ("The test set size is", "X:", X_test.shape, "y:", y_test.shape)

# Inner hold-out
X_train, X_val, y_train, y_val = train_test_split(X_dev, y_dev,
test_size=val_size, random_state=SEED)
# Print the size of the training and validation sets
print ("\nThe development set size is", "X:", X_train.shape, "y:", y_train.shape)
print ("The test set size is", "X:", X_val.shape, "y:", y_val.shape)
print()
ACC = []

C_list = [0.001, 0.01, 0.1, 1, 10, 100]

for c in C_list:
    clf = LogisticRegression(max_iter=1000, C=c).fit(X_train, y_train)
    y_pred_val = clf.predict(X_val)
    acc_val = accuracy_score(y_val, y_pred_val)
    print("Average ACC val set:", acc_val)
    ACC.append (acc_val)
bestC_index = np.argmax(ACC)
bestC = C_list[bestC_index]
print ("\nBest C: ", bestC, "ACC", ACC[bestC_index])

# Re-train the SVR model in the development test using the bestC hyperparameter
clf = LogisticRegression(max_iter=1000,C=bestC).fit(X_dev, y_dev)

# Apply the model to the test set and compute the MAE
y_pred_test = clf.predict(X_test)
acc_test = accuracy_score(y_test, y_pred_test)
print("Average ACC test set:", acc_test)

```